*The EAOS methodology defines a structured lifecycle for Designing, deploying, and governing enterprise-grade AI systems*

## Project: Standard Framework Project

# 3.3 Tool Config

### Phase: 3. Capability

### [산출물 ID: 3.3 Tool Config.docx]

**WBS Task**

3.3.1 Agent별 Tool 목록 및 권한 정의
3.3.2 Tool 호출 흐름 및 Error Handling 정의
3.3.3 Permission 설정 및 외부 연동 보안 기준

| | |
|---|---|
| 문서 번호 | 3.3 Tool Config.docx |
| 버전 | v1.0 |
| 작성일 | 2026 |
| 작성자 | KY Song |
| 분류 | CONFIDENTIAL |
| 적용 대상 | Standard Framework 프로젝트 전체 |

# Table of Contents

1. 개요 ..... 3
1.1 Tool 분류 체계 ..... 3
1.2 Tool 공통 인터페이스 ..... 3
1.3 Tool 호출 흐름 ..... 3
2. Internal Tools ..... 4
2.1 db_query — PostgreSQL 쿼리 실행 ..... 4
2.2 redis_cache — Redis 캐시 접근 ..... 5
2.3 internal_api — 내부 서비스 API 호출 ..... 6
3. Agent Tools ..... 7
3.1 llm_call — LLM 호출 ..... 7
3.2 agent_message — Agent 간 메시지 전달 ..... 8
4. External Tools ..... 9
4.1 slack_notify — Slack 알림 발송 ..... 9
4.2 email_send — 이메일 발송 ..... 9
4.3 webhook_call — 외부 Webhook 호출 ..... 10
5. Permission Matrix ..... 12
5.1 Agent별 Tool 접근 권한 ..... 12
5.2 권한 위반 처리 ..... 12
6. Error handling 전략 ..... 13
6.1 오류 분류 및 처리 기준 ..... 13
6.2 Circuit Breaker 설정 ..... 13
6.3 Retry 전략 ..... 13
7. Tool Config 환경별 설정 ..... 14
7.1 환경별 Tool 동작 차이 ..... 14
7.2 환경별 Timeout 설정 ..... 14
8. 산출물 체크리스트 ..... 15
10. 승인 (Approval) ..... 16

# 1. 개요

Tool 은 Agent 가 외부 시스템(DB, API, 파일 시스템, 알림 서비스)에 접근하는 **최하위 인터페이스**다. Agent 는 Tool 을 직접 호출하지 않고 반드시 **Skill → Tool** 순서로 경유한다.

> 💡 Tool 은 "무엇에 접근하는가"를 정의하고, Skill 은 "어떻게 사용하는가"를 정의한다. Tool Config 는 접근 권한, 연결 설정, Error Handling 의 단일 진실 출처(Single Source of Truth)다.

## 1.1 Tool 분류 체계

```
Tool 분류
├── Internal Tools     — 프로젝트 내부 시스템 (DB, Redis, 내부 API)
├── Agent Tools        — LLM 호출, Agent 간 통신
└── External Tools     — 외부 서비스 (알림, 이메일, Webhook)
```

## 1.2 Tool 공통 인터페이스

```
class ToolBase:
    tool_id: str
    tool_name: str
    version: str
    allowed_agents: list[str]
    requires_approval: bool
    timeout_seconds: int = 30
    retry_count: int = 3
    retry_delay_seconds: float = 1.0

    async def call(self, input: ToolInput) -> ToolOutput:
        raise NotImplementedError

    async def health_check(self) -> bool:
        raise NotImplementedError
```

## 1.3 Tool 호출 흐름

```
Agent
  ↓ Skill 경유
Skill Registry
  ↓ 권한 확인 + 승인 체크
Tool Registry
  ↓ Health Check
Tool
  ↓ 실행
  ├── 성공 → 결과 반환 + audit_log
  └── 실패 → Retry → 초과 시 Escalate + audit_log(failure)
```

# 2. Internal Tools

## 2.1 db_query — PostgreSQL 쿼리 실행

| 항목 | 값 |
|---|---|
| Tool ID | internal.db_query |
| 버전 | v1.0.0 |
| 허용 Agent | Back-A, Orch, Evol-A (읽기), Test-A (테스트 DB 전용) |
| 승인 필요 | 쓰기(INSERT/UPDATE/DELETE): **P1 필수** / 읽기: 불필요 |
| Timeout | 30 초 |
| Retry | 3 회 (지수 백오프) |

**연결 설정:**

```
# settings.py 경유 — 코드 내 직접 연결 정보 금지
DB_CONFIG = {
    "host": settings.POSTGRES_HOST,
    "port": settings.POSTGRES_PORT,
    "database": settings.POSTGRES_DB,
    "user": settings.POSTGRES_USER,
    "password": settings.POSTGRES_PASSWORD,
    "pool_size": 20,
    "max_overflow": 10,
    "pool_timeout": 30,
    "pool_recycle": 1800,
    "echo": settings.DB_ECHO,          # 프로덕션: False
    "echo_pool": settings.DB_ECHO,
}
```

**입력 스키마:**

```
class DbQueryInput(BaseModel):
    trace_id: UUID
    agent_id: str
    operation: Literal["SELECT", "INSERT", "UPDATE", "DELETE"]
    query: str              # SQLAlchemy ORM 쿼리 (raw SQL 금지)
    params: dict = {}
    use_transaction: bool = True

    @validator("operation")
    def write_requires_approval(cls, v, values):
        if v in ["INSERT", "UPDATE", "DELETE"]:
            # 호출 Skill 에서 P1 승인 확인 필수
            pass
        return v
```

**출력 스키마:**

```
class DbQueryOutput(BaseModel):
    trace_id: UUID
    rows_affected: int
```

```
    result: list[dict] | None
    execution_time_ms: float
    transaction_id: str | None
```

**Error Handling:**

| 오류 유형 | 처리 방법 |
|---|---|
| Connection Error | 3 회 Retry (1s → 2s → 4s) → Escalate to Arch |
| Query Timeout (30s) | 즉시 롤백 + Escalate |
| Integrity Error (FK, Unique) | 롤백 + 상세 오류 반환 (재시도 없음) |
| Permission Error | 즉시 차단 + 보안 알림 |

---

## 2.2 redis_cache — Redis 캐시 접근

| 항목 | 값 |
|---|---|
| Tool ID | internal.redis_cache |
| 버전 | v1.0.0 |
| 허용 Agent | 전체 |
| 승인 필요 | 불필요 |
| Timeout | 5 초 |
| Retry | 3 회 |

**연결 설정:**

```
REDIS_CONFIG = {
    "host": settings.REDIS_HOST,
    "port": settings.REDIS_PORT,
    "password": settings.REDIS_PASSWORD,
    "db": 0,
    "decode_responses": True,
    "socket_timeout": 5,
    "socket_connect_timeout": 5,
    "retry_on_timeout": True,
    "health_check_interval": 30,
}
```

**입력 스키마:**

```
class RedisCacheInput(BaseModel):
    trace_id: UUID
    operation: Literal["GET", "SET", "DELETE", "EXISTS", "EXPIRE", "INCR"]
    key: str
    value: str | None = None
    ttl_seconds: int | None = None    # SET 시 TTL 필수 권장

    @validator("key")
```

```
    def validate_key_format(cls, v):
        # 키 형식: "{prefix}:{identifier}" 필수
        if ":" not in v:
            raise ValueError("Redis key must follow ´{prefix}:{id}´ format")
        return v
```

**키 네임스페이스 규칙:**

| 접두사 | 용도 | TTL 기준 |
|---|---|---|
| session:{user_id} | 사용자 세션 | 24h |
| refresh_token:{user_id}:{jti} | Refresh Token | 30 일 |
| login_attempts:{email} | 로그인 실패 횟수 | 15 분 |
| approval:{approval_id} | 승인 요청 캐시 | 48h |
| agent_run:{trace_id} | Agent 실행 상태 | 1h |
| kpi:{metric_name}:{date} | KPI 메트릭 | 7 일 |
| rate_limit:{user_id}:{endpoint} | API Rate Limit | 1 분 |

## 2.3 internal_api — 내부 서비스 API 호출

| 항목 | 값 |
|---|---|
| Tool ID | internal.api_call |
| 버전 | v1.0.0 |
| 허용 Agent | Orch, Back-A, Test-A |
| 승인 필요 | 불필요 (내부 서비스) |
| Timeout | 15 초 |
| Retry | 3 회 |

**입력 스키마:**

```
class InternalApiInput(BaseModel):
    trace_id: UUID
    service: Literal["auth", "notification", "billing", "agent-runner"]
    method: Literal["GET", "POST", "PUT", "PATCH"]
    path: str
    payload: dict = {}
    headers: dict = {}     # Authorization 은 자동 주입 (직접 설정 금지)
```

# 3. Agent Tools

## 3.1 llm_call — LLM 호출

| 항목 | 값 |
|---|---|
| Tool ID | agent.llm_call |
| 버전 | v1.0.0 |
| 허용 Agent | 전체 |
| 승인 필요 | 불필요 (Skill 레벨에서 제어) |
| Timeout | 120 초 |
| Retry | 2 회 (Rate Limit 오류 시 60 초 대기) |

**연결 설정:**

```
LLM_CONFIG = {
    "provider": settings.LLM_PROVIDER,        # "anthropic" | "openai"
    "model": settings.LLM_MODEL,                # "claude-sonnet-4-5" 등
    "api_key": settings.LLM_API_KEY,          # settings 경유 필수
    "max_tokens": 8192,
    "temperature": 0.1,                         # Agent 일관성을 위해 낮게 설정
    "timeout": 120,
}
```

**입력 스키마:**

```
class LlmCallInput(BaseModel):
    trace_id: UUID
    agent_id: str
    system_prompt: str        # 3.1.1 System Prompt Spec 기준
    messages: list[dict]     # {"role": "user"|"assistant", "content": str}
    tools: list[dict] = [] # Function Calling 정의
    max_tokens: int = 4096
    temperature: float = 0.1

    class Config:
        # system_prompt 는 항상 3.1.1 정의된 프롬프트를 사용
        # 런타임 중 system_prompt 임의 변경 금지
        pass
```

**출력 스키마:**

```
class LlmCallOutput(BaseModel):
    trace_id: UUID
    content: str
    tool_calls: list[dict] | None
    input_tokens: int
    output_tokens: int
    latency_ms: float
    model_used: str
    finish_reason: str
```

**Cost 제어 규칙:**

| 항목 | 기준 |
|---|---|
| 최대 Input Tokens | 100,000 (초과 시 Orch 에 Escalate) |
| 최대 Output Tokens | 8,192 |
| 월별 비용 임계값 | settings.LLM_MONTHLY_BUDGET (초과 시 Monitor-A 알림) |
| 병렬 호출 최대 | 5 개 (초과 시 큐 대기) |

## 3.2 agent_message — Agent 간 메시지 전달

| 항목 | 값 |
|---|---|
| Tool ID | agent.send_message |
| 버전 | v1.0.0 |
| 허용 Agent | 전체 (Orch 경유 필수) |
| 승인 필요 | 불필요 |
| Timeout | 10 초 |

**목적:** Agent 간 직접 통신을 금지하고 반드시 Orch 를 경유하여 메시지를 전달한다.

```
class AgentMessageInput(BaseModel):
    trace_id: UUID
    from_agent: str
    to_agent: str
    via_orch: bool = True     # 항상 True — 직접 통신 금지
    message_type: Literal["task_assign", "result_return", "escalation", "approval_request"]
    payload: dict
    priority: Literal["low", "normal", "high", "critical"] = "normal"
```

> ⚠□ **Agent 간 직접 메시지 금지.** via_orch: False 설정 시 Tool Registry 가 호출을 차단한다.

# 4. External Tools

## 4.1 slack_notify — Slack 알림 발송

| 항목 | 값 |
|---|---|
| Tool ID | external.slack_notify |
| 버전 | v1.0.0 |
| 허용 Agent | Orch, Monitor-A |
| 승인 필요 | 불필요 (알림 발송) |
| Timeout | 10 초 |
| Retry | 3 회 |

**연결 설정:**

```
SLACK_CONFIG = {
    "bot_token": settings.SLACK_BOT_TOKEN,
    "channels": {
        "alerts": settings.SLACK_CHANNEL_ALERTS,
        "approvals": settings.SLACK_CHANNEL_APPROVALS,
        "incidents": settings.SLACK_CHANNEL_INCIDENTS,
    }
}
```

**입력 스키마:**

```
class SlackNotifyInput(BaseModel):
    trace_id: UUID
    channel: Literal["alerts", "approvals", "incidents"]
    urgency: Literal["low", "medium", "high", "critical"]
    title: str
    body: str
    mention_users: list[str] = []    # Slack User ID 목록
    action_url: str | None = None    # 승인 링크 등
```

**채널별 사용 기준:**

| 채널 | 사용 시점 | 멘션 대상 |
|---|---|---|
| alerts | KPI 임계값 초과, Drift 감지 | Monitor-A → 담당자 |
| approvals | P1 승인 요청 발생 | Orch → 승인 주체 |
| incidents | CRITICAL 이상, 보안 이벤트 | Orch → Business Owner |

---

## 4.2 email_send — 이메일 발송

| 항목 | 값 |
|---|---|
| Tool ID | external.email_send |

| 항목 | 값 |
|---|---|
| 버전 | v1.0.0 |
| 허용 Agent | Orch, Monitor-A |
| 승인 필요 | 불필요 (알림 발송) |
| Timeout | 15 초 |
| Retry | 3 회 |

**연결 설정:**

```
EMAIL_CONFIG = {
    "provider": settings.EMAIL_PROVIDER,    # "ses" | "sendgrid"
    "api_key": settings.EMAIL_API_KEY,
    "from_address": settings.EMAIL_FROM,
    "from_name": "Standard Framework AI System",
}
```

**입력 스키마:**

```
class EmailSendInput(BaseModel):
    trace_id: UUID
    to: list[str]                # 수신자 목록
    subject: str
    body_html: str
    body_text: str               # 텍스트 폴백
    priority: Literal["low", "normal", "high"] = "normal"
    attachments: list[dict] = []
```

---

## 4.3 webhook_call — 외부 Webhook 호출

| 항목 | 값 |
|---|---|
| Tool ID | external.webhook_call |
| 버전 | v1.0.0 |
| 허용 Agent | Orch, Back-A |
| 승인 필요 | **P1 필수** (G-06 외부 API 연동) |
| Timeout | 30 초 |
| Retry | 3 회 |

**입력 스키마:**

```
class WebhookCallInput(BaseModel):
    trace_id: UUID
    approval_id: UUID          # P1 승인 ID 필수
    url: str
    method: Literal["POST", "PUT"]
    headers: dict
```

```
    payload: dict
    signature_required: bool = True
    verify_ssl: bool = True  # False 금지 (프로덕션)
```

# 5. Permission Matrix

## 5.1 Agent 별 Tool 접근 권한

| Tool | Orch | Monitor-A | Back-A | Front-A | Test-A | Evol-A | UIUX-A |
|---|---|---|---|---|---|---|---|
| internal.db_query | ✓읽기 | ✓읽기 | ✓전체 | ✕ | ✓테스트 DB | ✓읽기 | ✕ |
| internal.redis_cache | ✓ | ✓ | ✓ | ✕ | ✓ | ✓ | ✕ |
| internal.api_call | ✓ | ✕ | ✓ | ✕ | ✓ | ✕ | ✕ |
| agent.llm_call | ✓ | ✕ | ✓ | ✓ | ✓ | ✓ | ✓ |
| agent.send_message | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| external.slack_notify | ✓ | ✓ | ✕ | ✕ | ✕ | ✕ | ✕ |
| external.email_send | ✓ | ✓ | ✕ | ✕ | ✕ | ✕ | ✕ |
| external.webhook_call | ✓ | ✕ | ✓(P1) | ✕ | ✕ | ✕ | ✕ |

## 5.2 권한 위반 처리

모든 권한 위반은 아래 순서로 처리된다.

> ① Tool Registry 가 권한 매트릭스 조회
> ② 허용되지 않은 Agent 감지 시 → 호출 즉시 차단
> ③ PermissionViolationError 발생
> ④ audit_logs 에 PERMISSION_VIOLATION 기록
> ⑤ Orch 에게 보안 이벤트 보고
> ⑥ 해당 Agent 세션 일시 중단 (Orch 판단)

# 6. Error handling 전략

## 6.1 오류 분류 및 처리 기준

| 오류 코드 | 분류 | Retry | Escalate | 감사 로그 |
|---|---|---|---|---|
| TOOL_TIMEOUT | 일시적 | ✓ 3 회 | 3 회 초과 시 | ✓ |
| TOOL_RATE_LIMIT | 일시적 | ✓ 대기 후 | - | ✓ |
| TOOL_CONNECTION_ERROR | 인프라 | ✓ 3 회 | 3 회 초과 시 | ✓ |
| TOOL_PERMISSION_ERROR | 보안 | ✗ | 즉시 | ✓ |
| TOOL_VALIDATION_ERROR | 입력 오류 | ✗ | - | ✓ |
| TOOL_NOT_FOUND | 설정 오류 | ✗ | 즉시 | ✓ |
| APPROVAL_REQUIRED | 승인 필요 | ✗ | 승인 요청 | ✓ |

## 6.2 Circuit Breaker 설정

```
CIRCUIT_BREAKER_CONFIG = {
    "db_query": {
        "failure_threshold": 5,    # 5 회 연속 실패 시 차단
        "recovery_timeout": 60,    # 60 초 후 복구 시도
    },
    "llm_call": {
        "failure_threshold": 3,
        "recovery_timeout": 30,
    },
    "external_webhook": {
        "failure_threshold": 3,
        "recovery_timeout": 120,
    }
}
```

## 6.3 Retry 전략

```
# 지수 백오프 + Jitter
def calculate_retry_delay(attempt: int, base: float = 1.0) -> float:
    delay = base * (2 ** attempt)              # 1s → 2s → 4s
    jitter = random.uniform(0, delay * 0.1) # 10% Jitter
    return min(delay + jitter, 30.0)          # 최대 30 초
```

# 7. Tool Config 환경별 설정

## 7.1 환경별 Tool 동작 차이

| Tool | local | dev | staging | production |
|---|---|---|---|---|
| internal.db_query | Docker PostgreSQL | Dev RDS | Staging RDS | Production RDS |
| internal.redis_cache | Docker Redis | Dev ElastiCache | Staging ElastiCache | Production ElastiCache |
| agent.llm_call | Claude API (제한) | Claude API | Claude API | Claude API |
| external.slack_notify | 콘솔 출력 | #test 채널 | #staging 채널 | 실제 채널 |
| external.email_send | 로컬 파일 저장 | 테스트 주소 | 스테이징 주소 | 실제 발송 |
| external.webhook_call | Mock 서버 | 실제 (P1 필수) | 실제 (P1 필수) | 실제 (P1 필수) |

## 7.2 환경별 Timeout 설정

```
TOOL_TIMEOUT_CONFIG = {
    "local": {
        "db_query": 60,       # 개발 환경은 여유 있게
        "llm_call": 180,
    },
    "production": {
        "db_query": 30,
        "llm_call": 120,
    }
}
```

# 8. 산출물 체크리스트

| # | 항목 | 완료 |
|---|---|---|
| 1 | Internal Tool 3 개 Config 정의 완료 | ☐ |
| 2 | Agent Tool 2 개 Config 정의 완료 | ☐ |
| 3 | External Tool 3 개 Config 정의 완료 | ☐ |
| 4 | Permission Matrix 전체 정의 완료 | ☐ |
| 5 | Error Handling 전략 정의 완료 | ☐ |
| 6 | Circuit Breaker 설정 정의 완료 | ☐ |
| 7 | 환경별 Config 차이 정의 완료 | ☐ |
| 8 | Architect 검토 완료 | ☐ |
| 9 | Back-H 권한 승인 완료 (G-04 통과) | ☐ |

> ✅ 9 개 항목 전체 완료 → **Phase 3 Tool 연동 가능**

# 10. 승인 (Approval)

| 구분 | 내용 | 서명 | 날짜 |
|---|---|---|---|
| 작성자 | | | |
| 검토자<br>(AI Architect) | | | |
| 승인자<br>(Business Owner) | | | |
| 버전 | v1.0 | | |